

Graduation Project — Task Documentation

Person 4 | RAG Pipeline + Vector Database

Backend • AI

Powered by LLM + RAG + Agentic AI

Overview

You build the memory of InsightX AI. When a report is confirmed, you take its text, split it into chunks, convert each chunk into a vector, and store it in Qdrant. When the Owner asks a question, you find the most relevant chunks and pass them to the AI Agent so it can answer accurately from real data.

What is RAG?

Task	Details
R — Retrieval	When a question arrives, search the Vector DB for the most relevant chunks.
A — Augmented	Add those chunks as context to the LLM prompt.
G — Generation	LLM generates an answer ONLY from the provided chunks, not from imagination.

Backend Tasks

ASP.NET Core | Semantic Kernel | Qdrant .NET SDK

1. Chunking Service

After a report is confirmed by the Manager, receive its full extracted text and split it into chunks of approximately 300-500 characters each. Each chunk should cover one topic. Do not cut sentences in the middle.

Example chunk from a report:

```
"January production reached 800 units against a target of 1000 units. Defect rate was 3%, within the acceptable 5% threshold."
```

2. Embedding Service

Convert each chunk to a vector (list of numbers) using an embedding model. Use Semantic Kernel's built-in text embedding via Azure OpenAI or OpenAI API. Each vector will have 1536 dimensions (OpenAI text-embedding-ada-002).

3. Qdrant Storage

Store each chunk in Qdrant with the following structure:

Field	Type	Description
id	UUID	Unique ID for this chunk
vector	float[]	1536-dimension embedding
text	string	Original chunk text
company_id	int	CRITICAL: used to filter per company
department_id	int	Department this chunk belongs to
report_id	int	Which report this came from
month	string	e.g. January 2025
year	int	e.g. 2025

4. Retrieval Service

When a question is received, convert the question to a vector, then search Qdrant for the top 5 most similar chunks. ALWAYS filter by company_id so a company never sees another company's data.

```
// Pseudocode
var questionVector = await embed(question);
var results = await qdrant.Search(
  collection: "reports",
  vector: questionVector,
  filter: new Filter {
    company_id == currentUser.CompanyId
  },
  limit: 5
);
```

5. API Endpoints

Task	Details
POST /rag/index	Receive confirmed report text. Run chunking + embedding + store in Qdrant.
POST /rag/retrieve	Receive a question. Return top 5 relevant chunks for that company.
DELETE /rag/{reportId}	Remove all chunks for a specific report from Qdrant.

Important: Namespace Isolation

Use a single Qdrant collection called 'reports' for all companies. Isolation is enforced by filtering on company_id in every query. Never query without the company_id filter — this is a security requirement.